Hybrid search and vector keyword → Keyword search + vector search.

Monday, April 27, 2026 11:50 PM

Vector search
- great at semantic meaning and context
- struggle with exact matches
  - search for API Key Auth → results → user login security.

→ can miss important keywords
→ sometimes user want results with exact keywords

Example:

TF Example: Chunk A says "Tesla's revenue growth is strong. Tesla reported record profits." vs Chunk B says "Tesla reported strong results." → "Tesla" query gets us Chunk A because it mentions "Tesla" twice

IDF Example: If "Tesla" appears in 100 chunks but "Cybertruck" appears in only 5 chunks → For the query "Tesla Cybertruck", those 5 chunks gets higher IDF score because it's rarer

Combined: For query "Tesla Cybertruck", a chunk with "Tesla's Cybertruck production" beats a chunk with "Tesla Tesla Tesla financial reports" because the rare term "Cybertruck" is more valuable than repeated common terms

BM25 is an improved version of TF-IDF that prevents keyword stuffing and handles chunk length better

Keyword search
- Looks for Exact Word
- Super Reliable → specific terms, names, model numbers, technical jargon.
- Still Dumb → Can't understand (car and automobile are the same.)

Why We Need Both: The Hybrid Approach

- Vector search catches semantic relationships and context
- Keyword search catches exact matches and specific terminology
- Together, they cover each other's weaknesses
- Think of it as having both a smart assistant (vectors) and a precise librarian (keywords) working together

The algo Behind BM25 → Best matching 25

(Term Frequency → how often it appears in the chunk.
Inverse document frequency → how rare across the chunk collection.)

TF example

A Tesla revenue is very strong. Tesla recorded profits
B Tesla reported strong Results

Q – "Tesla" → chunkA → Because it has tesla twice.

(Tesla → 100 chunks
cyber → 5 chunks)

"Tesla cyber" → 5 chunks Retrieved → high IDF score.

Case 1: Word is very rare, but appears once.
TF = low
IDF = high
TF-IDF = low × high = low

👉 Not important.

Case 2: Word is common everywhere.
TF = high
IDF = low
TF-IDF = low

👉 Still not important (stopwords like the, is).

Case 3: Word is frequent in this doc, rare elsewhere ⭐
TF = high
IDF = high
TF-IDF = HIGH

So here the thing that happens is RRF → is the part of the hybrid search. Not a separate Concept.

BM25 → Doc1, Doc3, Doc2
vector → Doc2, Doc1, Doc4

$w_1 = 0.5$
$w_2 = 0.5$

$$\sum w \left(\frac{1}{k + \text{rank-pos}} \right)$$

This is Applied And we get the final result chunks.